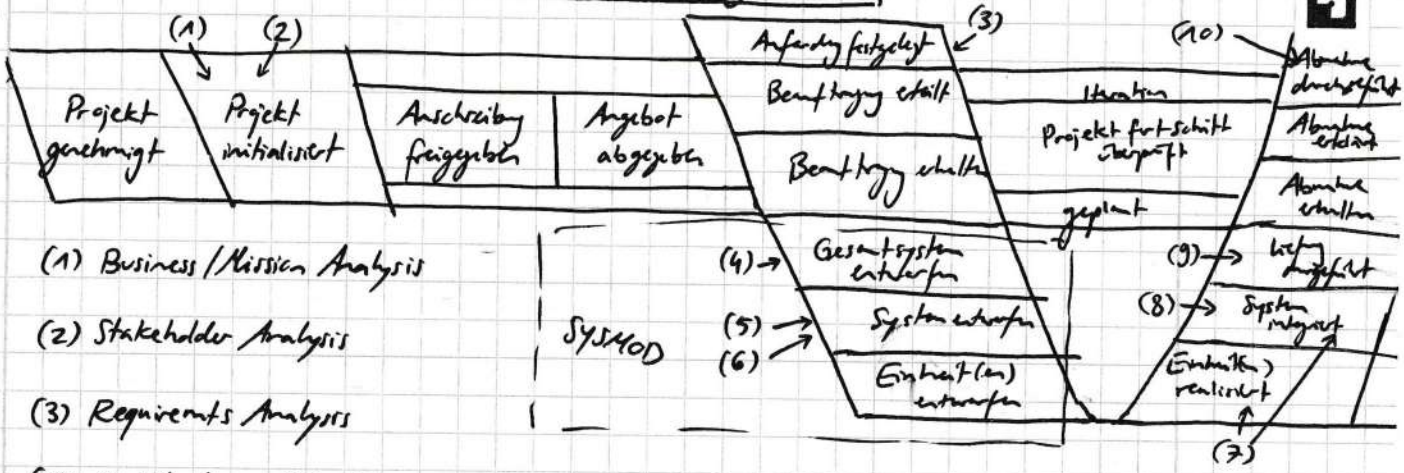
Chapter 7.1 : Technical System Engineering Process



(1) Business / Mission Analysis

(2) Stakeholder Analysis

(3) Requirements Analysis

(4) Architecture (5) Design (6) Analysis (7) Implementation, Integration

(8) Verification (9) Transition (10) Validation (11) Operation (12) Maintenance

(13) Disposal

→ Projekt abgeschlossen

(11-13)

(1[redacted] [redacted] **ConOps** Concept of Operation

→ intended way of business operation : how do we make money?

OpsCon System Operational Concept

→ from a user's perspective : why do I need this?

Company ConOps Why System OpsCon Why Customer

(2) - define stakeholder needs, drive requirements and organizational development of lifecycle concepts

(3) Requirements Definition Criteria :

- necessary
- bounded
- implementation independent
- unambiguous
- complete
- singular
- achievable
- verifiable
- confirming
- complete
- consistent
- feasible and affordable

(4) - arch alternatives are compared to find most suitable for customer, requirements and project constraints

- development of models for each alt. consisting of diffent views

(5) - extension of found arch into design by adding info until implementable
- eval of alt affirm or contend chosen arch

(6) - provides basis for aid in decision making across sys's life cycle:

→ LCC - life cycle cost

→ risk analysis - technical risk of sys throughout its lifetime

→ effectiveness analysis - degree of operational properties reached during lifetime

[redacted]

⋮

SysE
cont'd
31/01/26

SYSMOD Architecture Process

- architecture is sum of elements describing a system, that are fundamental

1. **Base Architecture (BA)**
   → constraints (technological, cultural, legislative) that define arch.
2. **Functional Architecture (FA)**
   → functions form primary goal of system
   → can be reused over different iterations, independent from technical solutions
   → can be found in use case, specified in activities
3. **Logical Architecture (LA)**
   → desc of technical concepts, to be used to provide successful technical concept for system's goals
4. **Physical Architecture (PA)**
   → specific physical blocks used to implement logical blocks, can be mixed with LA

Analysis Process

1. Define system idea and goals (desc basic principles, purpose of system)
2. Define base architecture (technological constraints and implications for project)
3. Requirements engineering including stakeholder analysis ← source of requirements
4. Modelling system context (system interfaces), (bdd), (ibd)
   → Sys interacts with external actors at boundary, exchanges obj via its interface
5. Define use cases that further describe functional requirements (uc)
   → free of redundancy, precisely described using activity diagrams
6. Identify domain blocks (ibd), (bdd)
   → aims to summarize domain blocks and their properties

BA
FA
} Requirement Specification (Lastenheft); what to do

LA
PA
} Target specification (Pflichtenheft); how to do it

[redacted]

[redacted]

SysE

01/02/26

THROs MiniSYSMOD Process

Phase 1: BA uses: SysML requirement diagram (req)

- [redacted] central requirements that define main purpose of system
  - ↳ opscon and conops
  - ↳ sub req to model constraints

Phase 2: FA uses: Use Case Diagram (uc), Activity Diagram (act)

- actor (users or other [redacted] systems) interaction with the system

↳ each use case gets an activity diag → describes [redacted] functional req from use case in detail

[redacted] **Domain Blocks** uses: Block Definition Diagramm (bdd)

- identify main domains [redacted] addressed in system
  - ↳ do we have the knowledge to build the systems?

Phase 3: LA

[redacted] external and logical interfaces uses Internal Block Diagram (ibd)

→ external intf modeled using parts, derived from use cases, which goods/info is provided from outside the system

→ [redacted] logical internal interfaces show info flow [redacted] between system properties

Phase 4: PA

- physical system decomp using Block Definition Diagram (bdd)
  - ↳ goal: [redacted] realize system using the model to clarify how to build system

further diagrams:
- Package Diag (packages and dependencies) (pkg)
- Parametric Diag (used to model parameter constraints) (par)
- State Machine Diag (more precise than act) (stm)
- Sequence Diag (better to model elem interactions than act) (sq)

[redacted]

finished duplicate on other page

Process Summary

1. Define system ideas and goals (req) – desc basic principles, sys purpose
2. Define base architecture (bdd), (ibd) – technological constraints, sys implications
3. Requirements engineering including (based on stakeholder analysis (uc), (req)
:

[redacted]

[redacted]

SysE

THROs MiniSYSMOD Process

01/02/26

Phase 1: BA uses: SysML requirement diagram (req)

- [redacted] central requirements that define main purpose of system
  - ↳ opscon and conops
  - ↳ sub req to model constraints

Phase 2: FA uses: Use Case Diagram (uc), Activity Diagram (act)

- actor (users or other [redacted] systems) interaction with the system

↳ each use case gets an activity diag → describes [redacted] functional req from use case in detail

[redacted] Domain Blocks uses: Block Definition Diagramm (bdd)

- identify main domains [redacted] addressed in system
  - ↳ do we have the knowledge to build the systems?

Phase 3: LA

[redacted] external and logical interfaces uses Internal Block Diagram (ibd)

→ external intf modeled using parts, derived from use cases, which goods/info is provided from outside the system

→ [redacted] logical internal interfaces show info flow [redacted] between system properties

Phase 4: PA

- physical system decomp using Block Definition Diagram (bdd)
  - ↳ goal: [redacted] how to build system realize system using the model to clarify

further diagrams:
- Package Diag (packages and dependencies) (pkg)
- Parametric Diag (used to model parameter constraints) (par)
- State Machine Diag (more precise than act) (stm)
- Sequence Diag (better to model elem interactions than act) (sq)

Chapter 8: Formal Verification

Verification is a proof that a given programme or system is behaving correctly according to it specification

Validation is an assessment of software or system qualification according to its intended use (customer expectations)

Requirements for sound verification techniques:
- fast - cheap
- useable - integrated
- safe/error-free - automated
- exhaustive - measuring
- subtle - measurable

Techniques:

Reviews - (Peer Review, Walkthrough, Inspection)
- manual with low completeness but easy to apply and effective during early development phases

Emulation - hardware configured to behave like system under test
→ e.g. during chip development using reprogram. chips followed by testing

Simulation - Model of sys executed in software, then tested

Test - test cases compared to expected behavior, does not typically proof correctness of program
→ can be automated, creative process

Formal Verification - is complete according to certain aspect, automated hard to apply and costs a lot of effort

Testing: done to uncover faults, not proof system/program correctness

→ System Test: black-box test against specification (also called: functional tests)

→ White Box Testing: during software development only, uses control or data flow measure coverage of a number of test cases to measure the quality of tests (??)

Formal Verification: based on mathematical precise model, process and rules

↳ Techniques: Model based Simulation / Testing, Model Checking, Computer assisted Proof

MBSI: software tool used to simulate system behavior for intended or user-defined scenarios, analyzed for an entire scenario rather than single test

pro: fast, easy, shows counter-examples

con: not measure for quality, only user-defined scenarios, not exhaustive

MB Testing : test cases [redacted] auto-derived from sys model, results analysed by correct state

pro: easy, fast, cover, uses coverage criteria

con: only exec representative, not exhaustive

CA Proof : [redacted] behavior modelled using logic formulas, correct behavior is property and sys is correct if correct property follows from sys formalization
- often used for software or software-intense [redacted] systems

pro: proof, freedom of faults, systems with infinite states suitable

con: slow, often manual since problem undecidable, often no solution

Model Checking : state space has to be finite since analysis is [redacted] exhaustive
→ better for control-intensive appl. rather than data-intensive

pro: all properties can be proven individ., "push button" tech, no creativity required, covers entire logical areas instead of single test cases

con: all model-based only methods only as good as their model
state space explosion leads to non-calculability quickly

State space and variables → **independent** : [redacted] can change freely between states, e.g. inputs

→ **dependent**: change between [redacted] two states but depends on independent var

→ mix of both

LTL – Linear (time) Temporal Logic

- LTL formula is true at specific state i
- specifies what happens on all paths formulated for one specific path

$X\varphi$: φ [redacted] true in next state

$F\varphi$: φ true at least once in the future

$G\varphi$: φ true for all states

$\varphi_1 U \varphi_2$: φ_1 [redacted] true until φ_2 is true

e.g. GFp → "For all paths, from every state, a state can be reached where p = true" happening infinitely often

SysE

01/02/26 cont'd.

CTL - Computation Tree Logic

- consists of formulas of propositional calculus in addition with temporal operators with quantification over all paths
- specifies what happens on all paths from the initial state

$AX\varphi$: on all paths, φ is true in next state

$AF\varphi$: —"— , φ true at least once in future

$AG\varphi$: —"— , φ true for all states

$A\varphi_1 U \varphi_2$: —"— , φ_1 true until φ_2 true

$EX\varphi$: at least one path, φ true next state

$EF\varphi$: —"— , φ true at least once in future

$EG\varphi$: —"— , φ true for all states

$E\varphi_1 U \varphi_2$: —"— , φ_1 true until φ_1 true

e.g. AFp means on all paths from initial state, p happens at least once

Difference LTL vs CTL: LTL can only express what happens on all paths

CTL good at branching-time inevitability/existence

CTL can only express that on every path p is eventually true

LTL good at [redacted] path-patterns over time

Typical properties checked

Liveness property: from every state, reach state where system operational

[redacted] **Reset property**: from every state, initial reachable

Safety property: condition holds on all paths for all states, specifying things that never occur simultaneously

Security property: on all paths, never reach a state that requires authorization without prior authorization

Model Checking Phases

Modelling Phase: system modelled using formal [redacted] model language typically state-based model
additionally, properties to be checked also formulated

Model Checking Phase: software executes model and checks correctness of model according to properties

Analysis Phase: if sys fulfills properties, it is correct
if sys deviates, model checker will find counterexamples
that shows path from ■ initial to erroneous state

Counterexamples can happen when a) model is wrong
b) CTL/LTL is wrong
c) system does not hold the condition
d) any of the above in combo

SpE

01/02/26 cont'd

Chapter 9: Maintenance and Reliability [redacted] Theory

Reliability is a quality attribute of a system giving the probability for a certain point in time that the system's functions are behaving as specified

Availability is a quality attribute of a system giving the fraction of the lifetime of a system where the primary system's functions can be used as specified

Safety is given if no unacceptable risks can emerge from the system, not quantified in general

→ implementing one of these will often reduce the other two

Maintenance is the process of executing tasks on a system in operation to maintain reliability, availability or safety depending on importance for [redacted] its operation

Types:

> **Reactive**: performed after failure occurs
→ low cost components with low failure consequences, failures immediately evident

> **Preventive**: performed after fixed time or usage intervals [redacted]
↳ goal to reduce failure probability
→ wear out comps, legally required inspections, comps with known [redacted] lifetime behavior

> **Corrective**: performed after fault but before loss of function
→ detectable faults, immediate shutdown not required, coordination with operational schedules

> **Condition-based**: triggered by observed condition of comp, requires inspection, measurements, monitoring
→ progressive degradation, early warning signs exist, avoidance of unnecessary preventive work

> **Predictive**: advanced form of condition-based using data analysis, trends and models to predict remaining useful life
→ critical comps, sensor-equipped systems, measurable degradation patterns

Strategy depends on 1. risk on system level for failure of that component

2. and on the nature of how the failure occurs or is observed

Component Classification

- system decomposed into manageable parts (is called "item")
- consequences of failure analysis (COFA) performed to identify:
  - > function of item (what does it do?)
  - > functional failures and dominant failure modes (in what ways can it fail?)

COFA logic tree:
- > recognizability of failure mode (how easy to notice? in what way can it be noticed / found?)
- > effect on system level (hazard)
- > consequence on system level (harm) depending on quality criteria
- > component classification

Run-to-failure component : not safety-critical or critical for availability, failure evident during normal operation, [redacted] inexpensive to replace, no long downtime
→ reactive maintenance
info req.: failure impact, cost, frequency and repair time

also req. ↗

Economic components : not critical (safety, availability), evident failure, but costly to replace (more so than keeping it alive)
→ preventative or corrective maintenance

Commitment component: not directly evident, do not affect service quality, usually fulfill external commitments and legal obligations
→ preventative maintenance
info req.: failure impact.

Potentially critical components: directly evident but not directly affects [redacted] safety or central functionality under normal circumstances
→ preventative or condition-based maintenance
info req.: failure impact under abnormal conditions, [redacted] P/F interval (potential to functional failure, how long until total failure)

same info req but under normal conditions ↗

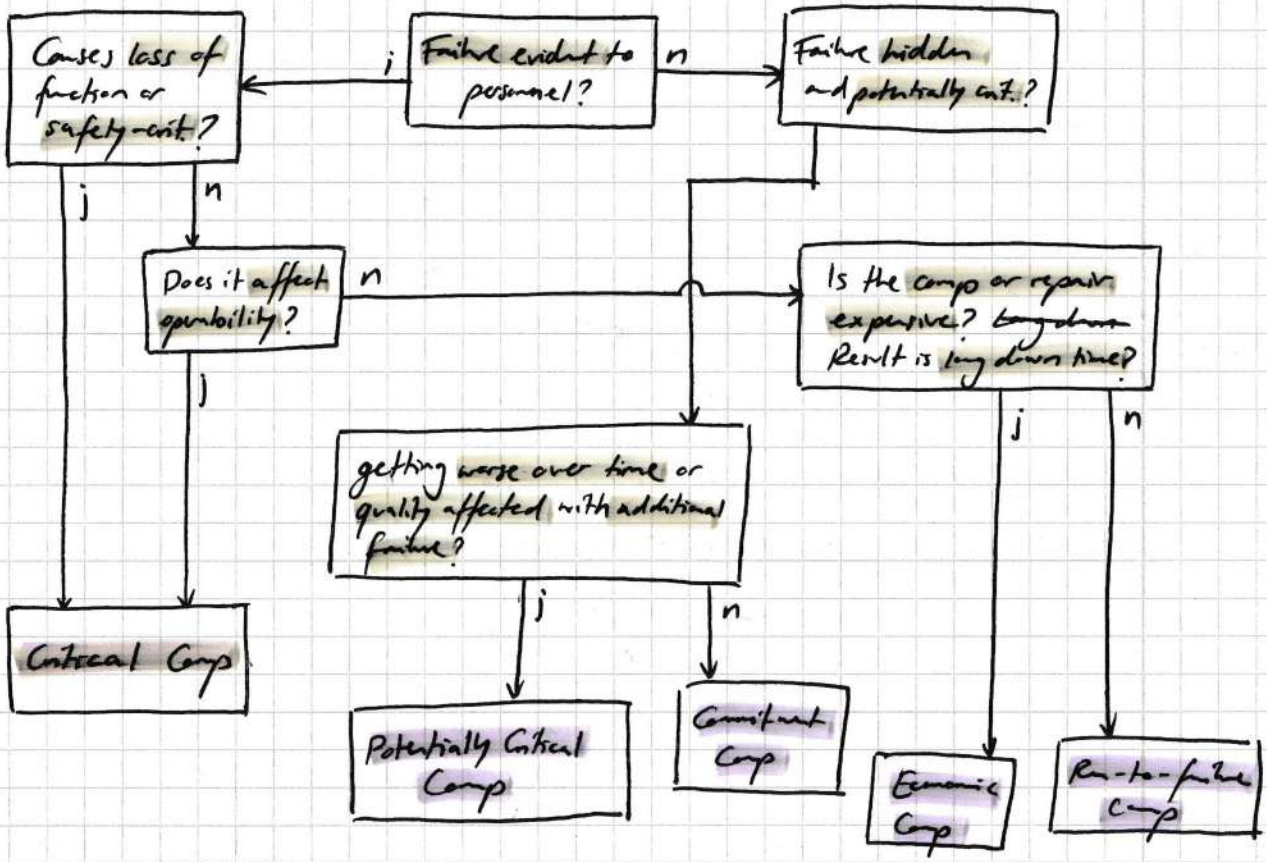
Critical component : directly affects safety or central functions, needs to be proactively maintained
→ preventative, corrective, condition-based or predictive maintenance

SysE

01/02/26

Ch9 contd.

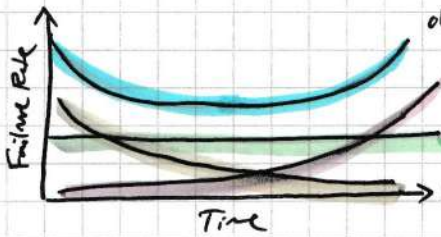
COFA logic tree



Worksheet [redacted]:

Component	Functions	Failures	Dominant Failure Mode	Evident?	(Hazard) System Effect	(Harm) Consequences on System Level	Component Classification

> Failure Distribution usually bathtub-shaped: → first failure due to production faults



→ then random hardware faults

→ then more failures due to aging effects

> Failures based on random faults result in an exponential distribution of lifetime

[redacted] MTTF – Mean Time To Failure → mean time until a failure appears after system operation start/restart

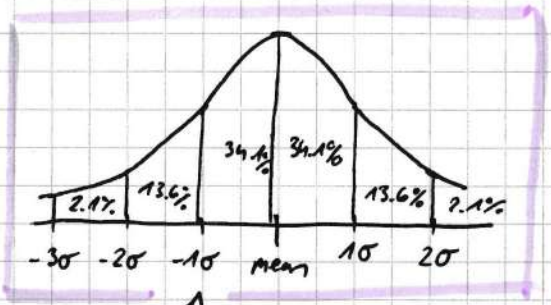
MTTR – Mean Time To Repair → mean time that it takes to repair the system

MTBF – Mean Time Between Failures → mean time between two failures, inverse is failure rate of a system

$MTBF = MTTF + MTTR = \frac{1}{\lambda}$

Six Sigma

- exchanging the comp at MTTF has a 50/50 chance of failure before replacement according to standard normal distribution



- exchanging 1 standard deviation (one sigma) before that reduces it by 34.1% to 15.9% and so on, e.g. $3\sigma = 50 - 34.1 - 13.6 - 2.1 = 0.2$
- six sigma results in a chance of 0.0000002%

Six Sigma is defined as 3.4ppm (Parts per Million) : $P(Z > 4.5) \approx 3.4 \times 10^{-6}$

$$t_{replace} = MTTF - 4.5\sigma$$

$$\sigma = \sqrt{\frac{\Sigma(X - \overline{X})^2}{n}}$$

X is failure time
$\overline{X}$ is MTTF
n is total failures

$$MTTF = \frac{\Sigma(X)}{n}$$

Sigma = standard deviation

- Constant failure rates means very high variance (comp constantly fails)

Predictive Maintenance Techniques

> Acoustic Monitoring : detect internal or external leaks in valves or tubes

> Vibration Monitoring and Analysis : in rotating machinery used to detect wear, imbalances, cracks, trend

> Infrared Monitoring : detect loose electronic connections, hot spots in motor windings or relay coils

> Oil analysis : detect gear failures or failures in transformers

> Radiography : detect subsurface weld flaws or failures in valves

> Magnetic particle inspection or eddy current testing : detect surface cracks by magnetic fields

> Ultrasonic testing

> Liquid penetrant : use dye to detect surface cracks

> Motor current signature analysis

> Boroscope inspections : internal inspections e.g. for tubes and valves

> Diagnostics for motor or air operated valves

SysE
cold
01/02/26

Redundancy

Types:

Passive Redundancy: parallel exec, does not detect failures but masks them and prevents them from propagating through rest of system

Active Redundancy: failure detection mechanism, reacts on failure e.g. by recalculating result or redoing actions

→ **Homogenous Redundancy** if redundant modules identical; can deal with random hardware faults

→ **Diverse Redundancy** if redundant modules different but implementing same specification; can cope with systematic faults

> **Hot Spare**: module active [redacted] all the time

> **Cold Spare**: module only activated when required

Triple Modular Redundancy (TMR) uses voter and selects identical results as valid result
passive redun

benefit: vast increase in reliability over short time

drawback: voter is single point of failure, reliability lower if mission time increases

[redacted] **Duplication with Comparison**
active redun

failures detected more reliably

decreases reliability via false negatives

→ tradeoff between reliability and safety

rate of undetected failures decreases

Cold Redundancy cannot be modelled using prev methods

lifetime of coldspare typically starts to decrease when activated